Design handoff review: what each asset needs to scale

Backend review of `design-handoff/` (Design Brief v2, 27 Oct 2026). Base kit lives in `base-kit/`.

Issues found in the handoff (fixed in base-kit)

1. **Fonts load from Google Fonts at render time** (`base.css @import`). On a server this is slow, can fail silently and fall back to a system font. Fixed: Instrument Sans/Serif self-hosted in `base-kit/fonts/` (OFL), declared in `tokens/tokens.css`.
2. **Legacy CSS variable names** (`--purple` holds green, `--violet`, `--lilac`, `--gold`). Fixed: semantic `--fl-*` tokens; legacy names kept as aliases so old templates still work.
3. **Missing file**: `certificate.html` references `../brand/mark-cream.png` (pattern layer) which doesn't exist, so the pattern renders empty. Dropped from the kit template; Design can supply it later.
4. **Stale rule in design-brief.md**: "Purple gradient field, gold for brand marks" should read green field / yellow marks. Flag to Design.
5. **render.py launches a new Chromium per image.** Fine for 4 files, too slow at scale. Kit renderer reuses one browser (12 exports ≈ 11 s; one merchant's cert + badge card ≈ 2.8 s cold, well under 1 s warm).
6. **Certificate has no verification QR**, and the profile link is printed only as text, so a screenshot can be faked. Added a "Scan to verify" QR slot pointing at the verify page.
7. **Templates fill fields by element ID with no blank mode.** Kit templates use `data-f` slots plus `?blank=1`, so the same file produces both the blank base PNG and the personalised render.

Per-asset decisions

Asset	Type	What's needed to replicate at scale	Owner
floate-logo.png, wordmark-*, icon-*	Brand layer	Already clean transparent cut-outs. Copied to `base-kit/layers/`. SVG still wanted from Design.	Shared (Frontend for web)
Verified seal (badge.html)	Static layer	Not personalised, so render once. Exported at 1080/512/256/128/64 px. Same seal for every merchant.	Frontend (UI badge), Backend (embedded in certs)
Verified certificate (1080x1920)	**Generated per merchant**	Blank base PNG + parametrised template (name, biz, loc, date, FLOATE ID, profile link, verify QR, check chips, tier). Rendered by the credential service.	Backend
Badge card (1080x1080)	**New, generated per merchant**	Square share card: seal + business + FLOATE ID + verify QR. Made for WhatsApp Status/IG, where a tall certificate is awkward.	Backend
Verify QR	**Generated per merchant**	Python `qrcode`, ECC M, #0E3B12 on white, encodes `https://floate.xyz/v/<FID>`.	Backend
Buyer QR (qr-buyer-whatsapp.png)	Static layer	One fixed wa.me link. Keep as is. Regenerate only if the number or prefill changes.	Shared
Square template (1080x1080)	Marketing template	Blank base + text/photo slots (tag, headline with one `<em>`, body, photo). Batch-rendered for campaigns and product boosts.	Backend renders, Frontend/Marketing fill copy
A5 market flyer	Print template	Blank base + headline/sub/pidgin/QR slots. City-specific QRs (UTM-tagged wa.me links) can be batch-rendered.	Backend renders, Marketing distributes
Backgrounds	Base layer	Status, square and A5 green fields, with and without logo/ghost icon, plus a cream reading panel.	Frontend + Marketing
Design tokens	Spec	`tokens/design-tokens.json` (source of truth) + `tokens/tokens.css`.	Shared

Open questions (carry to founder)

- Merchant profile URL format (placeholder `floate.xyz/m/<slug>`) and verify URL (proposed `floate.xyz/v/<FID>`).
- FLOATE ID format (placeholder `FLT-<CITY>-#####`): 3-letter city code list needed.
- What basic vs Verified Plus checks, since chips must match actual checks. Kit default: Real stall, Contact confirmed.
- Do buyer and merchant bots share +234 911 516 2081?

Spec: Floate Credential Service (certificate + badge generation)

v0.1 · 27 Oct 2026 · Backend · Prototype: `base-kit/src/generate_credentials.py`

Purpose

When a merchant passes verification (free basic) or pays for Verified Plus, generate their personalised certificate, badge card and verify QR automatically. Store the files, send them on WhatsApp/Telegram, and back them with a public verify page that can't be faked by editing a screenshot.

Triggers

Event	Source	Action
`merchant.verified` (basic)	Verification workflow sets `merchants.verification_status = verified`	Generate tier `basic`
`payment.succeeded` for Verified Plus	Payment webhook (Paystack/Flutterwave), signature checked, idempotent on `payment_ref`	Upgrade tier to `plus`, regenerate
`merchant.updated` (name, biz, loc, slug)	Admin/merchant edit	Regenerate, new version, old files kept
`merchant.revoked` / Plus expired	Admin or expiry cron	Mark credential `revoked`/`expired`. Verify page shows that status. Files are not deleted.

Events go onto a queue (e.g. Redis/BullMQ or SQS). Workers consume them. The bot never renders inline.

Inputs (job payload)

```
{ "fid": "FLT-ONT-00142", "merchant_id": "uuid", "name": "Chidinma Okafor",
  "biz": "Chidi Fabrics & Lace", "loc": "Onitsha Main Market, Anambra",
  "slug": "chidi-fabrics", "verified_on": "2026-10-27", "tier": "basic|plus",
  "checks": ["Real stall", "Contact confirmed"], "channel": "whatsapp|telegram", "chat_id": "..." }
```

Validation: name ≤ 40 chars, biz ≤ 48, loc ≤ 60 (template shrinks long names, hard cap rejects beyond). Strip HTML. `checks` must come from the verification record, never from free text.

Processing

1. Build verify URL `https://floate.xyz/v/<FID>` and QR (ECC M, #0E3B12 on white).
2. Fill `certificate.html` and `badge-card.html` via `data-f` slots. Render with a **warm pool** of headless Chromium pages (Playwright). Wait for `window.__ready` and `document.fonts.ready`, then screenshot `#c`.
3. Compute `content_hash` over (fid, name, biz, loc, verified_on, tier). If the hash and `template_version` are unchanged, skip the render (idempotent).
4. Upload, write a DB row, emit `credential.issued`.

Outputs

File	Size	Use
`<FID>-certificate.png`	1080x1920	WhatsApp Status / download
`<FID>-badge-card.png`	1080x1080	Status, IG, profile share
`<FID>-qr.png`	~740 px	Merchant prints at stall
`<FID>-manifest.json`	—	fid, verify_url, template_version, content_hash, files
Static seal (not per merchant)	64–1080 px	Web badge, served from CDN

Optional later: PDF certificate (print) from the same HTML via `page.pdf()`.

Storage

- Object storage (S3 / Cloudflare R2 / Supabase Storage): `credentials/<FID>/v<n>/<file>`. Public-read via CDN, immutable, cache 1 year. Each regeneration is a new version.
- Table `credentials`: `id, merchant_id, fid, tier, version, template_version, content_hash, status(active|revoked|expired), issued_at, expires_at, cert_url, card_url, qr_url`. Index on `fid`.
- No personal data beyond what's printed on the certificate. Phone numbers are never printed.

Delivery

WhatsApp Cloud API: upload media, then send an image message with a caption ("You're verified on Floate ✦ Share this on your Status") plus the verify link. Telegram: `sendPhoto`. Send via a template message if the 24 h session window has closed.

Verify-link page `floate.xyz/v/<FID>`

- Server-rendered, fast on 3G, no login. Content: Floate logo, seal, business name, market/location, tier (Basic / Verified Plus), verified-on date, status (green **Active**, red **Revoked**, grey **Expired**), the checks passed, a "Chat on WhatsApp" CTA to the merchant via the Floate bot, and a "Report this seller" link.
- Unknown FID: a "Not a Floate verified merchant" page with a warning.
- API: `GET /api/v1/verify/:fid → {fid, biz, loc, tier, status, verified_on, checks, profile_url}`. Rate-limited, cacheable 5 min, purged on status change.
- OG tags with badge-card image so shared links preview nicely.
- Page UI is built by **Frontend** from tokens. Backend owns the API and the status logic.

Performance and limits

Warm pool of 4 pages per worker ≈ 4–6 renders/s. 10k merchants is a few minutes of compute. Retries ×3 with backoff, then a dead-letter queue. Alert if the p95 render time exceeds 3 s.

Security

Payment webhooks signature-verified. Admin-only regenerate/revoke endpoints. FIDs are sequential per city, so the verify page shows only public fields. Later: add an HMAC short code under the QR to resist enumeration.

Open

Profile/verify URL format, FID city codes, what Plus checks, Plus duration/renewal, payment provider.

Handoff list: Frontend vs Backend

27 Oct 2026 · All paths relative to the Floate Backend project.

→ Frontend (website / UI)

Item	Path	Use
Design tokens	`base-kit/tokens/design-tokens.json`, `tokens.css`	Single source for colours, type, radius, shadow. Map into Tailwind/CSS vars.
Fonts (self-host)	`base-kit/fonts/*.ttf` + `OFL.txt`	Instrument Sans + Serif. Convert to woff2 for web.
Logo, wordmark, icon layers	`base-kit/layers/floate-logo.png`, `wordmark-{yellow,green,white}.png`, `icon-*.png`	Header, favicon, app icons.
Verified seal	`base-kit/layers/seal-verified-{64,128,256,512,1080}.png`	Badge beside merchant names, profile pages, search results.
Backgrounds	`base-kit/backgrounds/*`	Hero/section backgrounds, OG images.
Buyer QR	`base-kit/layers/qr-buyer-whatsapp.png`	"Start on WhatsApp" blocks.
Verify page UI	spec §Verify-link in `docs/02-credential-service-spec.md`	Build `floate.xyz/v/<FID>` against `GET /api/v1/verify/:fid`.
Merchant profile page	placeholder `floate.xyz/m/<slug>`	Shows seal + Plus badge, links to verify page.
Verified Plus purchase UI	—	Pricing/checkout screen. Calls the Backend payment endpoint.
Template previews	`base-kit/templates/*.png`, `base-kit/samples/*`	Reference for "your certificate" previews in the UI.

Stays with Backend

Item	Path
Credential service (queue worker, render pool, storage, delivery)	`docs/02-credential-service-spec.md`, prototype `base-kit/src/generate_credentials.py`
Parametrised templates (certificate, badge card, square, flyer)	`base-kit/src/*.html`, `fill.js`
Kit renderer (blank exports)	`base-kit/src/render_kit.py`
QR generation	`base-kit/src/qr.py`

Item	Path
Verify API + credential status (revoke/expire)	spec
Payment webhook → Plus upgrade trigger	spec
Batch renders for marketing (square posts, city flyers with UTM QRs, product boosts)	`square.html`, `flyer.html`

→ Design (feedback)

- Fix the stale "purple/gold" rule in design-brief.md.
- Supply SVG logo and the missing `mark-cream.png` pattern, if still wanted.
- Approve the added "Scan to verify" QR on the certificate and the new 1080 badge card.